LAB MANUAL

ON

INTERNET OF THINGS

DEPARTMENT OF ELECTRONICS AND COMMUNICATIONENGINEERING
(NAAC A+, NBA ACCREDITATED)
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES (A)
VISAKHAPATNAM

PREPARED BY:

PROF. P. MURUGUPANDIYAN DR. G. PRASANNA DR. N. SWATHI

Table of Contents

1. Introduction to Internet of Things (IOT)

2. Arduino in IoT

3.Arduino Uno

4. Arduino IDE (Integrated Development Environment)

5. Installation of Arduino Software (IDE)

6. Practical Experiments on Arduino and NodeMCU for designing IOT applications

Practical 1 Basic electrical connections and LED blinking

Practical 2 Basic digital input using push button switch

Practical 3 Basic analog input using LDR module and serial transmission of data to host PC

Practical 4 Basic analog output demo by varying LED brightness using Arduino code

Practical 5 Serial reception demo by varying LED pattern using serial monitor of Arduino

Practical 6 Basic humidity and temperature data transfer to thingspeak cloud analytics platform using node MCU

1.Internet of Things (IOT)

Introduction: IOT stands for “Internet of Things”. The IOT is a name for the vast collection of “things” that are being networked together in the home and workplace (up to 20 billion by 2020 according to Gardner, a technology consulting firm).

Characteristics of the IOT

Networking

These IOT devices talk to one another (M2M communication) or to servers located in the local network or on the Internet. Being on the network allows the device the common ability to consume and produce data.

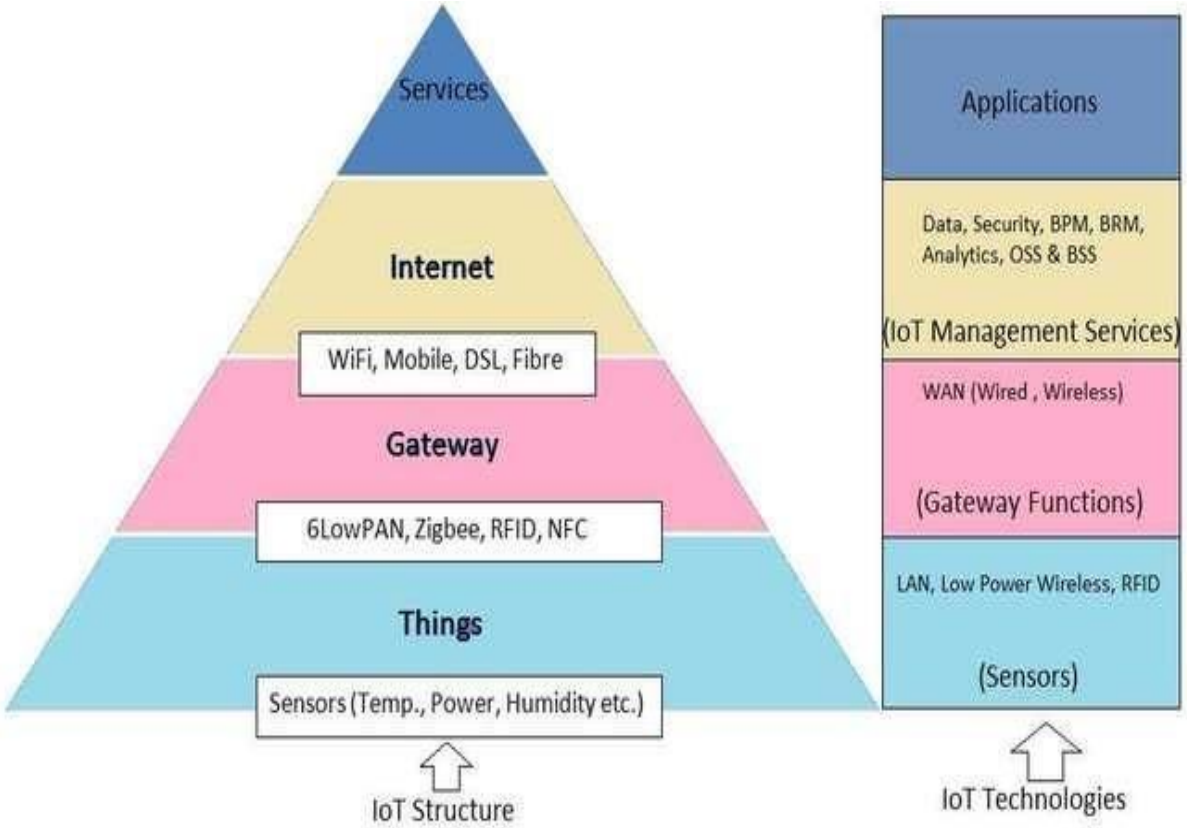
Sensing

IOT devices sense something about theirenvironment.

Actuators

IOT devices that do something. Lock doors, beep, turn lights on, or turn the TV on

Communications in IoT



Communications are important to IOT projects. In fact, communications are core to the whole genre. There is a trade-off for IOT devices. The more complex the protocols and higher the data rates, the more powerful processor needed and the more electrical power the IOT device will consume.

TCP/IP base communications (think web servers; HTTP-based commutation (like REST servers); streams of data; UDP) provide the most flexibility and functionality at a cost of processor and electrical power.

Low-power Bluetooth and Zigbee types of connections allow much lower power for connections with the corresponding decrease in bandwidth and

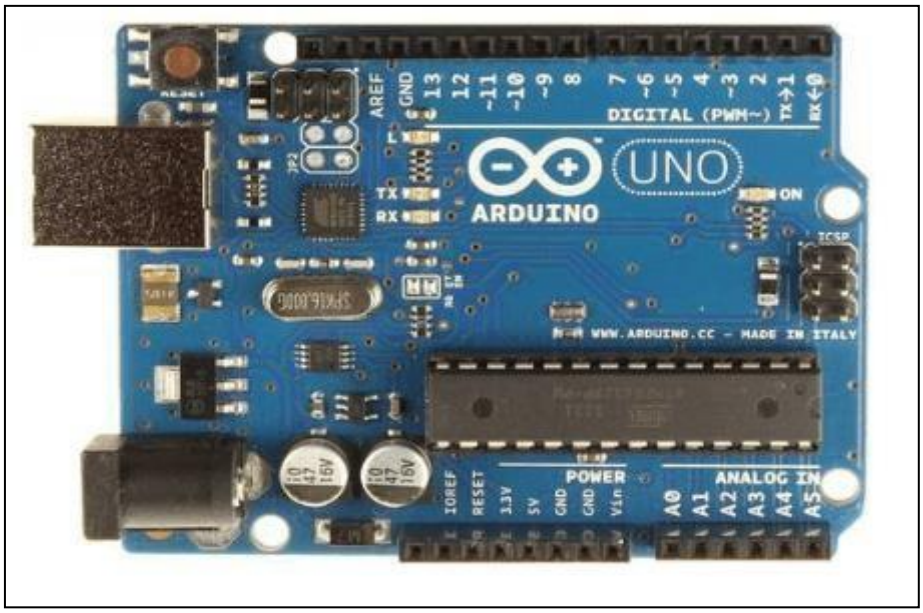
functionality. IOT projects can be all over the map with requirements for communication flexibility and data bandwidth requirements.

2. Arduino in IoT

In IoT applications the Arduino is used to collect the data from the sensors/devices to send it to the internet and receives data for purpose of control of actuators.

3.Arduino Uno

Introduction: The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller and developed by Arduino.cc. The board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable. It can be powered by the USB cable or by an external 9-volt battery, though it accepts voltages between 7 and 20 volts. The word "uno" means "one" in Italian and was chosen to mark the initial release of Arduino Software.



Features of the Arduino

1. Arduino boards are able to read analog or digital input signals from different sensors and turn it into an output such as activating a motor, turning LED on/off, connect to the cloud and many other actions.

2. The board functions can be controlled by sending a set of instructions to themicrocontroller on the board via Arduino IDE.
3. Arduino IDE uses a simplified version of C++, making it easier to learn to program.
4. Arduino provides a standard form factor that breaks the functions of the micro-controller into a more accessible package.

4.Arduino IDE

(Integrated DevelopmentEnvironment)

Introduction: The Arduino Software (IDE) is easy-to-use and is based on the Processing programming environment. The Arduino Integrated Development Environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in functions from C and C++. The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. This software can be used with any Arduinoboard.

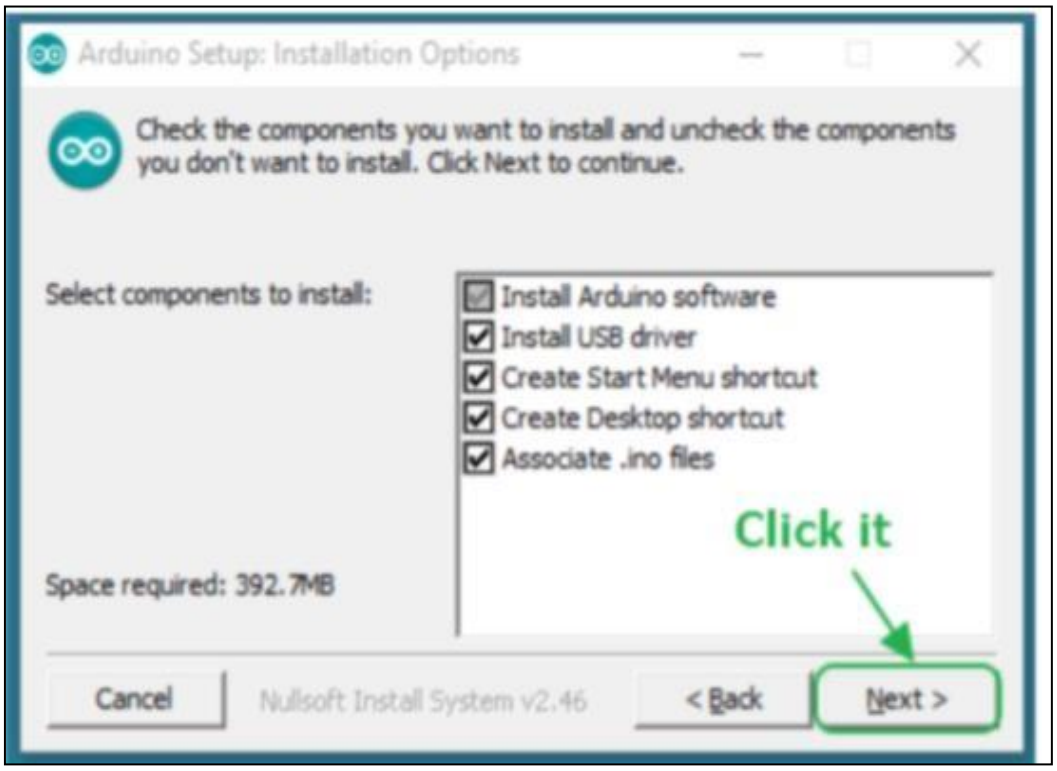
The Arduino Software (IDE) – contains:

- A text editor for writing code
- A message area
- A text consoles
- A toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

5. Installation of Arduino Software (IDE)

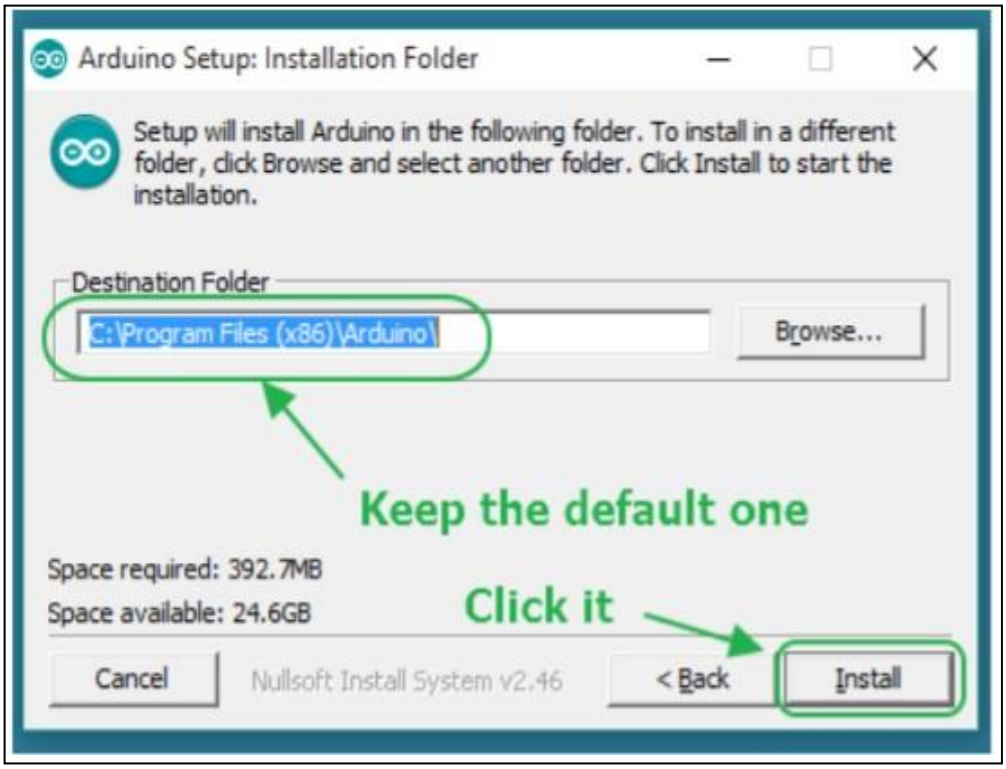
Step1: Downloading

- To install the Arduino software, download this page: http://arduino.cc/en/Main/Software and proceed with the installation by allowing the driver installation process.



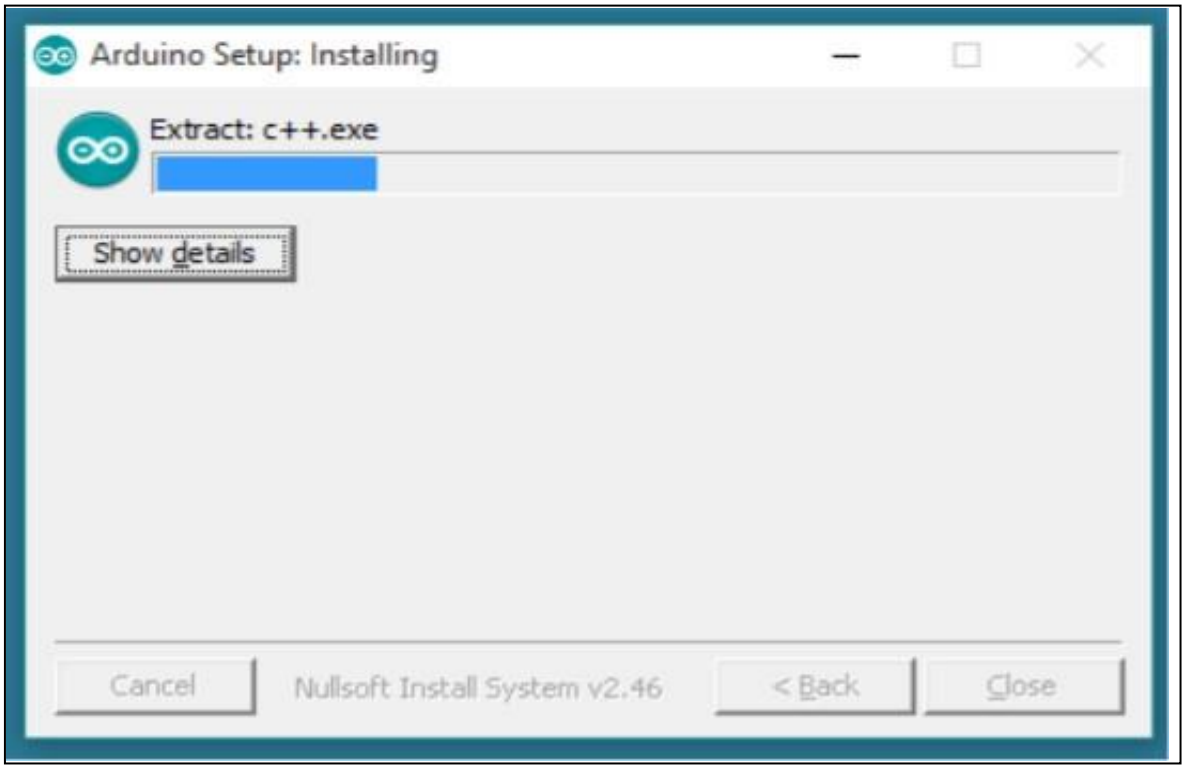
Step 2: Directory Installation

- Choose the installation directory.



Step 3: Extraction of Files

- The process will extract and install all the required files to execute properly the Arduino Software (IDE)

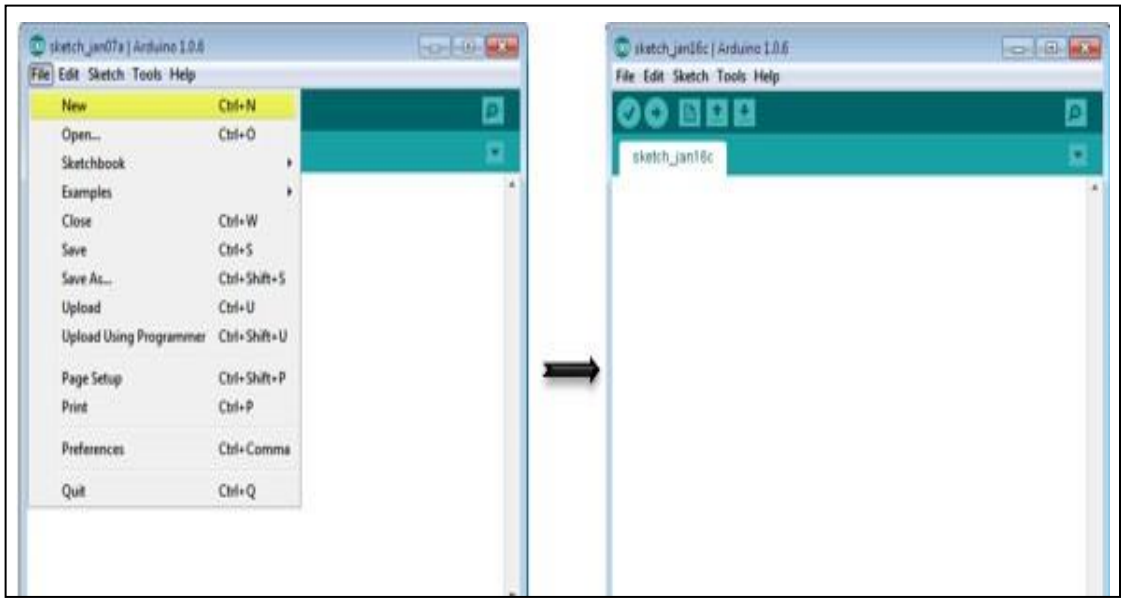


Step 4: Connecting the board

- The USB connection with the PC is necessary to program the board and not just to power it up. The Uno and Mega automatically draw power from either the USB or an external power supply. Connect the board to the computer using the USB cable. The green power LED (labelled PWR) should go on.

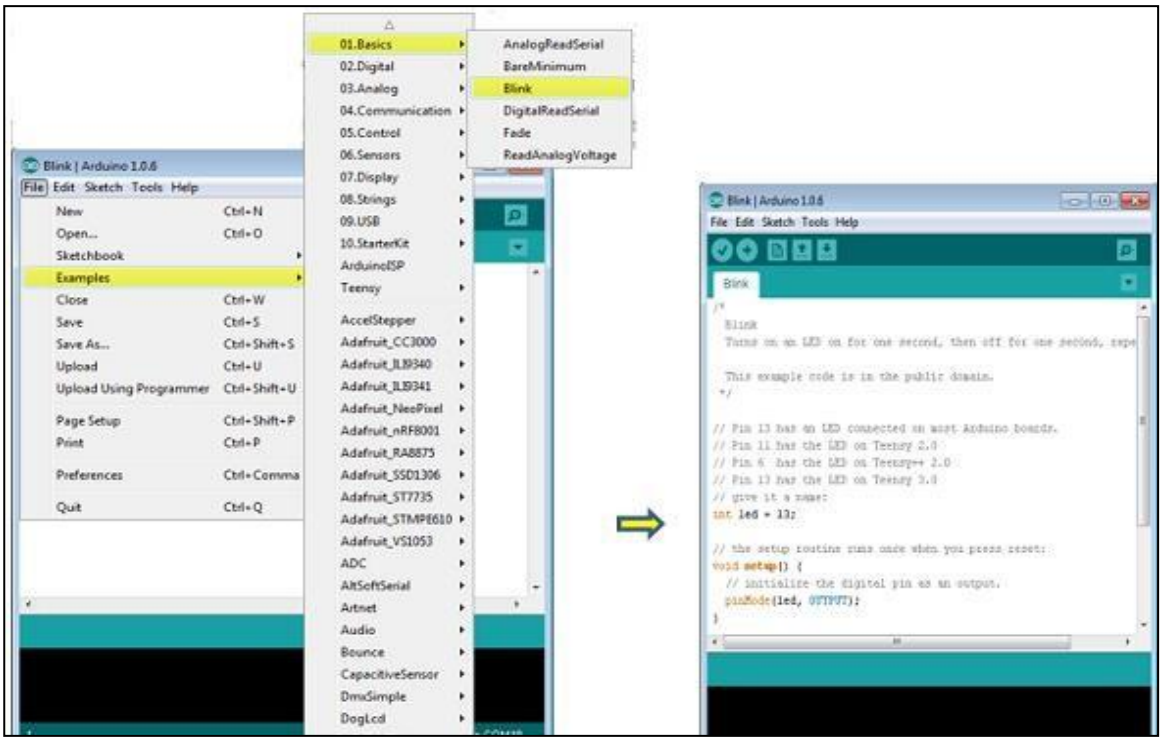
Step 5: Working on the new project

- Open the Arduino IDE software on your computer. Coding in the Arduino language will control your circuit.
- Open a new sketch File by clicking on New.



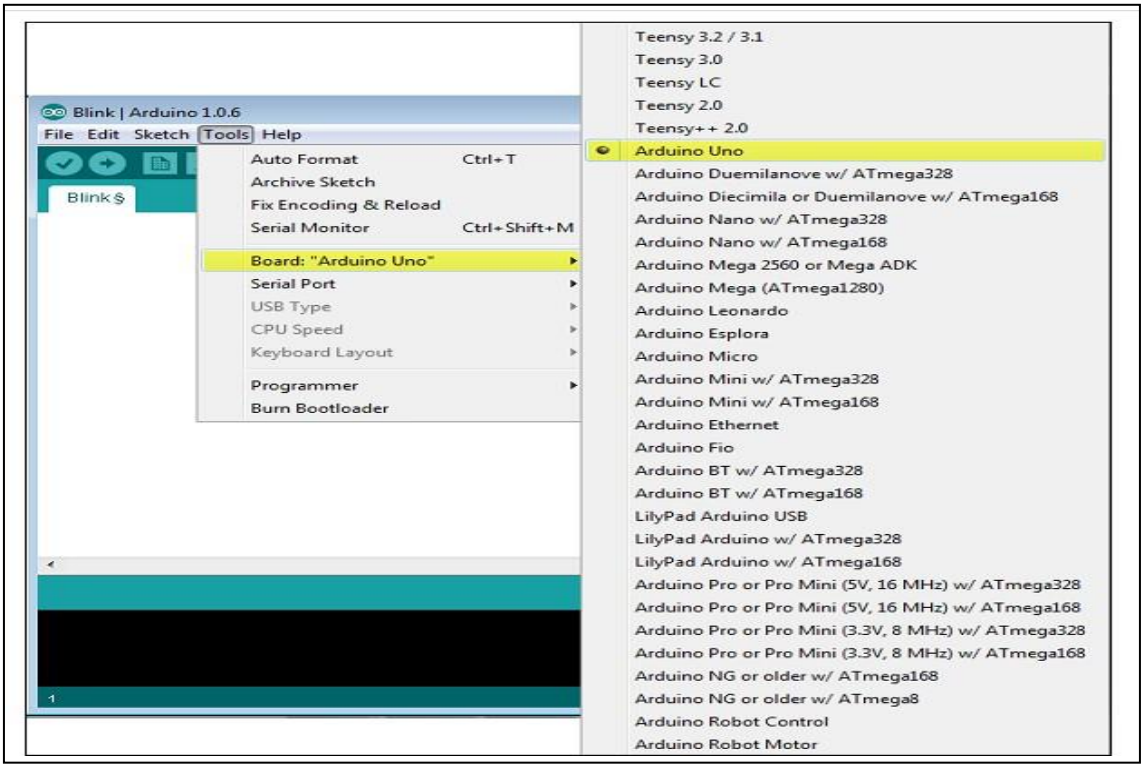
Step 6: Working on an existing project

- To open an existing project example, select File → Example → Basics → Blink.



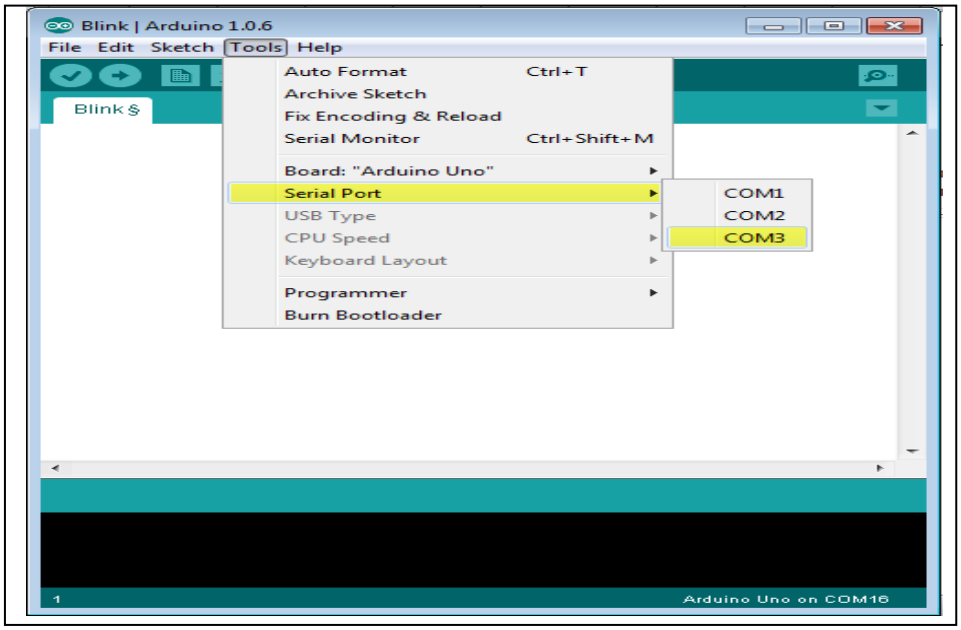
Step 7: **Select your Arduino board.**

- To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.
- Go to Tools → Board and select your board.



Step 8: Select your serial port

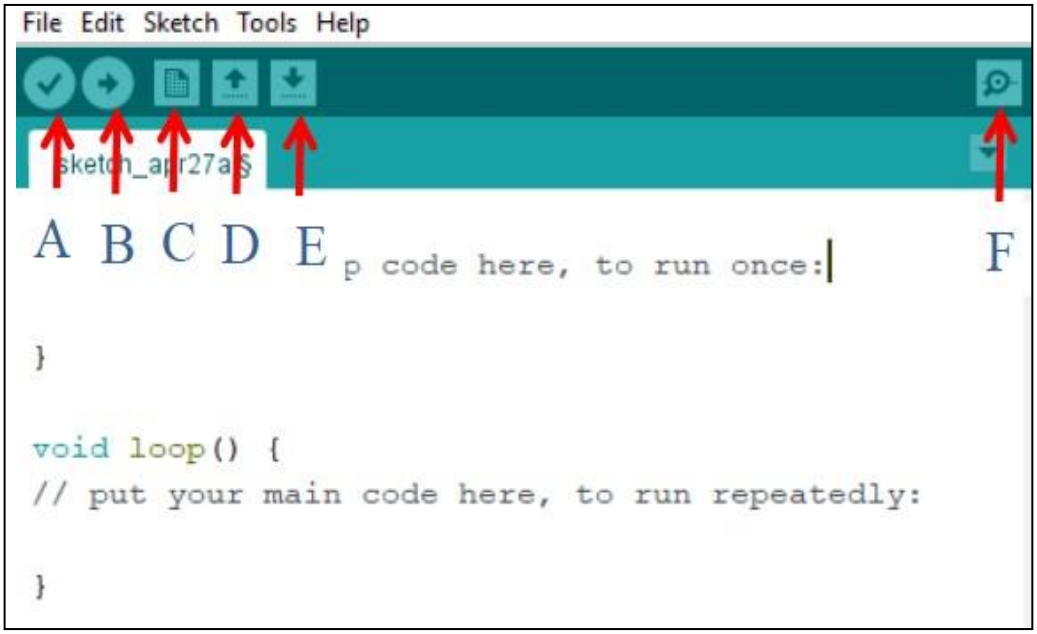
- Select the serial device of the Arduino board.
- Go to Tools → Serial Port menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports).
- To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the boardand select that serial port.



Step 9: Upload the program to your board.

- Click the "Upload" button in the environment.
- Wait a few seconds; you will see the RX and TX LEDs on the board, flashing.
- If the upload is successful, the message "Done uploading" will appear in thestatus bar.

A	Verify
B	Upload
C	New
D	Open
E	Save
F	Serial Motor


File Edit Sketch Tools Help
sketch_apr27a §
A B C D E
F
p code here, to run once:
}
void loop() {
// put your main code here, to run repeatedly:
}

File Edit Sketch Tools Help
BareMinimum §
Sketch Name
Global Area
void setup() {
Setup Area
}
void loop() {
Loop Area
}

6. Practical Experiments on Arduino and NodeMCU for designing IOT applications

Experiment 1

Basic LED Blinking Program Using Arduino Uno

Objective

To learn how to use an Arduino Uno microcontroller to blink an LED ON and OFF at regular intervals.

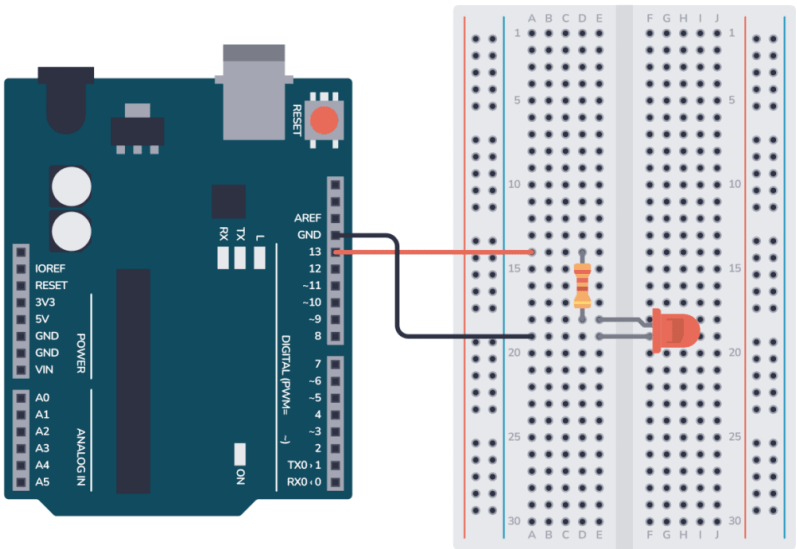
Components Needed

- Arduino Uno board
- USB cable (Type A to B)
- Breadboard
- LED (any color)
- 220 or 1 K-ohm resistor
- Jumper wires
- Computer with Arduino IDE installed

Pre-lab Preparation

1. Install the Arduino IDE on the lab computer if it is not already installed. The software can be downloaded from the official Arduino website.
2. Familiarize with the components: Arduino Uno, breadboard, LED, resistor, and jumper wires.

Circuit Diagram



Procedure

Step 1: Setting Up the Hardware

1. **Connect the LED to the Breadboard**
   - Insert the LED into the breadboard. The longer leg (anode) is the positive side, and the shorter leg (cathode) is the negative side.
2. **Connect the Resistor**
   - Connect one end of the 220 or 1 K-ohm resistor to the cathode of the LED.
   - Connect the other end of the resistor to a ground (GND) rail on the breadboard.
3. **Wire the LED to the Arduino**
   - Use a jumper wire to connect the anode of the LED to digital pin 13 on the Arduino Uno.
   - Use another jumper wire to connect the ground rail on the breadboard to one of the GND pins on the Arduino.

Step 2: Writing the Code

1. **Open the Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the Blink Code**
   - the code on the left side of the page is entered into the Arduino IDE:

```
// LED Blink Example

// Pin number where the LED is connected
const int ledPin = 13;

// The setup function runs once when you press reset or power the board
void setup() {
  // Initialize the digital pin as an output
  pinMode(ledPin, OUTPUT);
}

// The loop function runs over and over again forever
void loop() {
  digitalWrite(ledPin, HIGH);   // Turn the LED on (HIGH is the voltage level)
  delay(1000);                  // Wait for a second
  digitalWrite(ledPin, LOW);    // Turn the LED off by making the voltage LOW
  delay(1000);                  // Wait for a second
}
```

3. **Save the Code**
   - Save the sketch with an appropriate name, such as "LED_Blink".

Step 3: Uploading the Code

1. **Connect the Arduino Uno to the Computer**
   - Use the USB cable to connect the Arduino Uno to your computer.
2. **Select the Board and Port**
   - In the Arduino IDE, go to `Tools > Board` and select `Arduino Uno`.
   - Then, go to `Tools > Port` and select the port that your Arduino is connected to (it usually looks like `COM3` or higher on Windows or `/dev/tty.usbmodem` on Mac).
3. **Upload the Code**
   - Click the upload button (right arrow) in the Arduino IDE to compile and upload the code to the Arduino Uno.

Step 4: Observing the Results

1. **Verify the LED Blinks**
   - After the code is uploaded, observe the LED. It should blink on for one second and off for one second, repeatedly.

Troubleshooting

- If the LED does not blink:
  - Ensure all connections are secure.
  - Verify that the LED is correctly oriented (long leg to pin 13, short leg to ground via the resistor).
  - Check the resistor value; it should be 220 ohms.
  - Make sure the correct board and port are selected in the Arduino IDE.
  - Verify that the code is correctly uploaded (no errors during compilation).

Conclusion

An Arduino Uno was successfully programmed to blink an LED at regular intervals. This basic exercise is an introduction to the fundamental concepts of digital output and timing, which are essential for more complex projects.

References

- Arduino Getting Started Guide
- Arduino Blink Example

Experiment 2

LED Blinking on Button Press Using Arduino Uno (Basic Digital Input Demo)

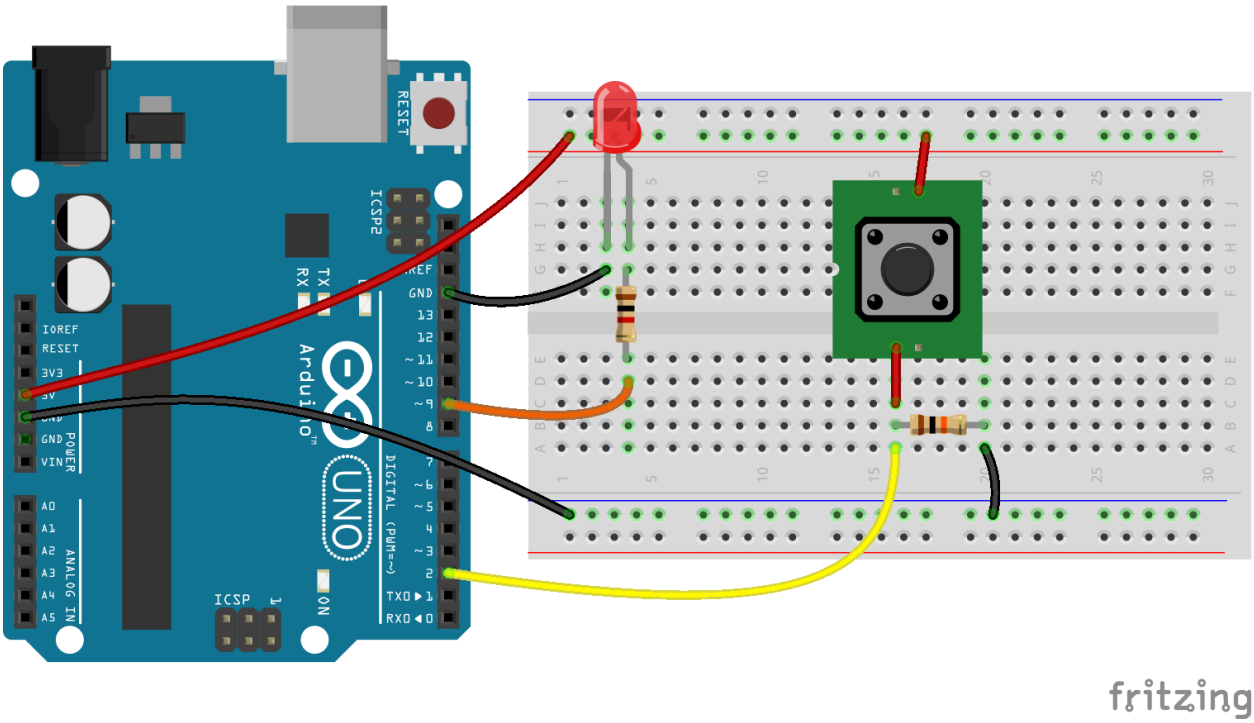
Objective

To demonstrate how to use a digital input (pushbutton) to control a digital output (LED) on an Arduino Uno.

Components Needed

- Arduino Uno board
- USB cable (Type A to B)
- Breadboard
- LED (any color)
- 220-ohm resistor
- Pushbutton
- 10k-ohm resistor
- Jumper wires
- Computer with Arduino IDE installed

Circuit Diagram



Procedure

Step 1: Setting Up the Hardware

1. **Connect the LED to the Breadboard**
   - Insert the LED into the breadboard. The longer leg (anode) is the positive side, and the shorter leg (cathode) is the negative side.
2. **Connect the Resistor to the LED**
   - Connect one end of the 220-ohm resistor to the cathode of the LED.
   - Connect the other end of the resistor to a ground (GND) rail on the breadboard.
3. **Wire the LED to the Arduino**
   - Use a jumper wire to connect the anode of the LED to digital pin 13 on the Arduino Uno.
   - Use another jumper wire to connect the ground rail on the breadboard to one of the GND pins on the Arduino.
4. **Connect the Pushbutton to the Breadboard**
   - Place the pushbutton across the middle of the breadboard, ensuring each pair of legs is on separate rows.
5. **Connect the Pushbutton to the Arduino**
   - Connect one side of the pushbutton to the 5V rail on the breadboard.
   - Connect the other side of the pushbutton to digital pin 4 on the Arduino.
6. **Add a Pull-Down Resistor**
   - Connect one end of the 10k-ohm resistor to the ground rail on the breadboard.
   - Connect the other end of the resistor to the same row as the pushbutton connected to digital pin 4.

Step 2: Writing the Code

1. **Open the Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the Button-Controlled Blink Code**
   - Enter the code given on the left into the Arduino IDE:

```
// Button Controlled LED Blink

const int buttonPin = 2;   // Pin where the pushbutton is connected

const int ledPin = 13;     // Pin where the LED is connected

// Variable to store the button state

int buttonState = 0;
```

```
// The setup function runs once when you press reset or power the board
void setup() {
  // Initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
  // Initialize the button pin as an input
  pinMode(buttonPin, INPUT);
}
void loop() {
  // Read the state of the pushbutton value
  buttonState = digitalRead(buttonPin);
  // Check if the pushbutton is pressed
  // If it is, the buttonState is HIGH
  if (buttonState == HIGH) {
    // Blink the LED on and off
    digitalWrite(ledPin, HIGH);  // Turn the LED on
    delay(500);                  // Wait for half a second
    digitalWrite(ledPin, LOW);   // Turn the LED off
    delay(500);                  // Wait for half a second
  } else {
    // Ensure the LED is off when the button is not pressed
    digitalWrite(ledPin, LOW);
  }
}
```

1. **Save the Code**
    - Save the sketch with an appropriate name, such as "Button_LED_Control".

Step 3: Uploading the Code

1. **Connect the Arduino Uno to the Computer**
    - Use the USB cable to connect the Arduino Uno to your computer.
2. **Select the Board and Port**
    - In the Arduino IDE, go to `Tools > Board` and select `Arduino Uno`.
    - Then, go to `Tools > Port` and select the port that your Arduino is connected to (it usually looks like `COM3` or higher on Windows or `/dev/tty.usbmodem` on Mac).
3. **Upload the Code**
    - Click the upload button (right arrow) in the Arduino IDE to compile and upload the code to the Arduino Uno.

Step 4: Observing the Results

1. **Press the Pushbutton**
    - When you press the pushbutton, the LED should turn on and turn off repeatedly.
    - When you release the pushbutton, the LED should turn off.

Troubleshooting

- If the LED does not respond to the button press:
    - Ensure all connections are secure and correct.
    - Verify that the pushbutton is correctly oriented and connected.
    - Check the resistor values; the pull-down resistor should be 10k ohms, and the LED resistor should be 220 ohms.
    - Make sure the correct board and port are selected in the Arduino IDE.
    - Verify that the code is correctly uploaded (no errors during compilation).

Conclusion

An Arduino Uno is successfully programmed to blink an LED when a pushbutton is pressed. This exercise is an introduction to the fundamental concepts of digital input and output, which are essential for more complex projects.

Experiment 3:

Measuring Light Intensity Using an LDR Module with Arduino (Basic Analog Input)

Objective

To measure light intensity using a Light Dependent Resistor (LDR) module and an Arduino Uno, and display the digitized values on the Serial Monitor.

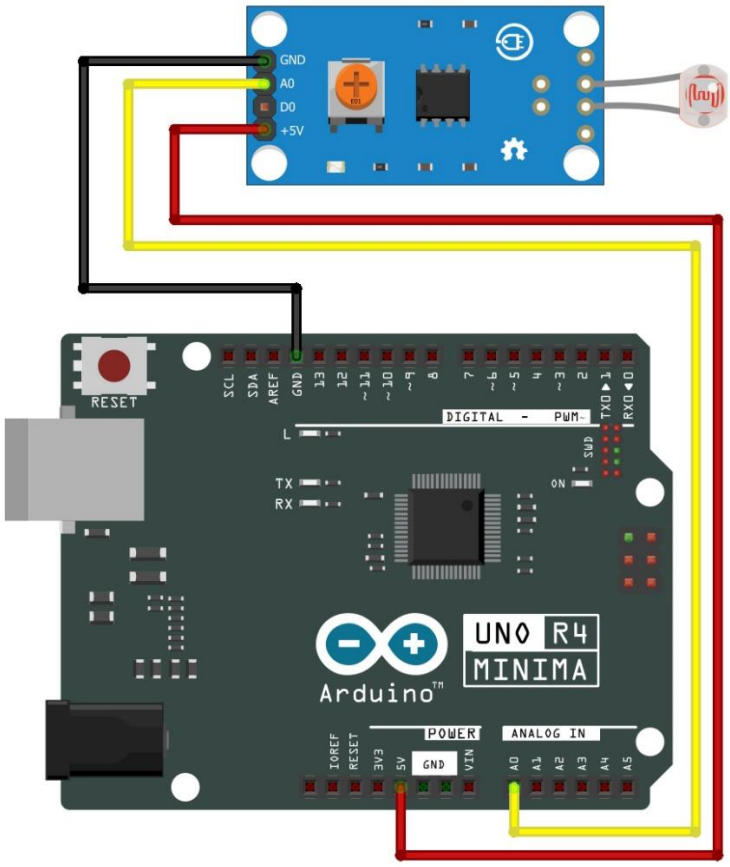
Components Needed

- Arduino Uno board
- USB cable (Type A to B)
- Breadboard
- Light Dependent Resistor (LDR)
- 10k-ohm resistor
- Jumper wires
- Computer with Arduino IDE installed

Pre-lab Preparation

1. Install the Arduino IDE on your computer if it is not already installed. The software can be downloaded from the official Arduino website.
2. Familiarize yourself with the components: Arduino Uno, breadboard, LDR, resistor, and jumper wires.

Circuit Diagram



Procedure

Step 1: Setting Up the Hardware

1. **Connect the LDR to the Breadboard**
   - Place the LDR on the breadboard.
2. **Create a Voltage Divider Circuit**
   - Connect one leg of the LDR to the 5V rail on the breadboard.
   - Connect the other leg of the LDR to an empty row on the breadboard.
   - Connect one end of the 10k-ohm resistor to the same row where the LDR is connected.
   - Connect the other end of the 10k-ohm resistor to the ground (GND) rail on the breadboard.
   - Connect a jumper wire from the junction of the LDR and the resistor (same row) to the A0 analog input pin on the Arduino.
3. **Power Connections**
   - Connect a jumper wire from the 5V pin on the Arduino to the 5V rail on the breadboard.

- Connect another jumper wire from one of the GND pins on the Arduino to the GND rail on the breadboard.

Step 2: Writing the Code

1. **Open the Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the LDR Reading Code**
   - Enter the following code into the Arduino IDE:

```
// LDR Light Intensity Measurement Example

// Define the LDR pin

const int ldrPin = A0;

void setup() {

  // Initialize serial communication at 9600 bits per second

  Serial.begin(9600);

}

void loop() {

  // Read the analog value from the LDR

  int ldrValue = analogRead(ldrPin);

  // Print the value to the Serial Monitor

  Serial.print("LDR Value: ");

  Serial.println(ldrValue);

  // Wait for 500 milliseconds before the next reading

  delay(500);

}
```

1. **Save the Code**
   - Save the sketch with an appropriate name, such as "LDR_Light_Intensity_serial".

Step 3: Uploading the Code

1. **Connect the Arduino Uno to the Computer**
   - Use the USB cable to connect the Arduino Uno to your computer.
2. **Select the Board and Port**
   - In the Arduino IDE, go to `Tools > Board` and select `Arduino Uno`.
   - Then, go to `Tools > Port` and select the port that your Arduino is connected to (it usually looks like `COM3` or higher on Windows or `/dev/tty.usbmodem` on Mac).
3. **Upload the Code**
   - Click the upload button (right arrow) in the Arduino IDE to compile and upload the code to the Arduino Uno.

Step 4: Observing the Results

1. **Open the Serial Monitor**
   - In the Arduino IDE, go to `Tools > Serial Monitor` or press `Ctrl + Shift + M`.
2. **View the LDR Values**
   - The Serial Monitor will display the LDR values read by the Arduino. These values represent the light intensity, with higher values indicating higher light levels and lower values indicating lower light levels.

Troubleshooting

- If no values are displayed on the Serial Monitor:
  - Ensure all connections are secure and correct.
  - Verify that the LDR and resistor are correctly oriented and connected.
  - Make sure the correct board and port are selected in the Arduino IDE.
  - Ensure the Serial Monitor baud rate is set to 9600 to match the `Serial.begin(9600)` in the code.
  - Verify that the code is correctly uploaded (no errors during compilation).

Conclusion

light intensity is successfully measured using an LDR module and an Arduino Uno, and the digitized values are displayed on the Serial Monitor. This exercise is an introduction to the fundamental concepts of analog input and serial communication, which are essential for more complex sensor-based projects.

Experiment 4:

Demonstrating Basic Analog Output from Arduino Uno Using LED Brightness Control from Stored Sequence of Values

Objective

To control the brightness of an LED using Pulse Width Modulation (PWM) from a stored sequence of values on an Arduino Uno.

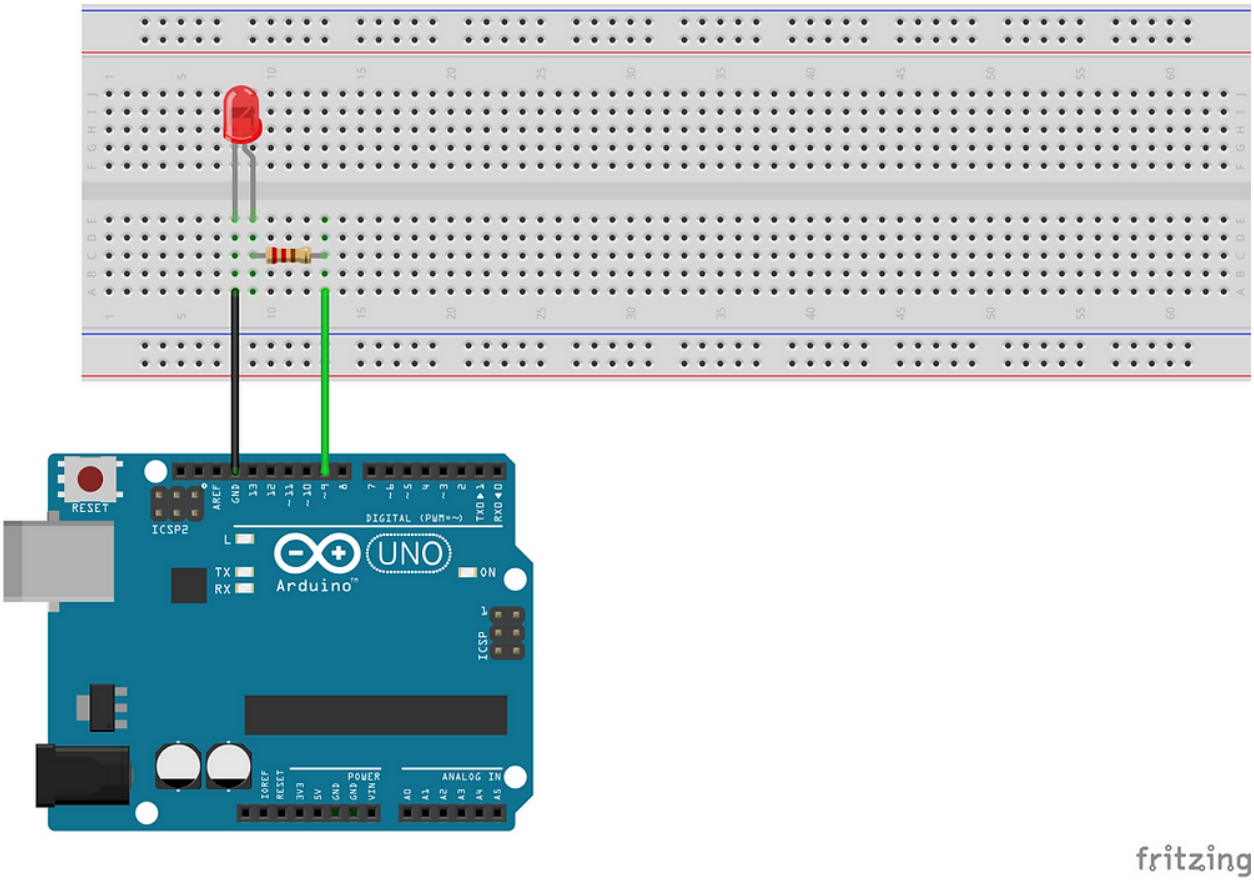
Components Needed

- Arduino Uno board
- USB cable (Type A to B)
- Breadboard
- LED (any color)
- 220-ohm resistor
- Jumper wires
- Computer with Arduino IDE installed

Pre-lab Preparation

1. Install the Arduino IDE on your computer if it is not already installed. The software can be downloaded from the official Arduino website.
2. Familiarize yourself with the components: Arduino Uno, breadboard, LED, resistor, and jumper wires.

Circuit Diagram



Procedure

Step 1: Setting Up the Hardware

1. **Connect the LED to the Breadboard**
   - Place the LED on the breadboard. The longer leg (anode) is the positive side, and the shorter leg (cathode) is the negative side.
2. **Connect the Resistor to the LED**
   - Connect one end of the 220-ohm resistor to the cathode of the LED.
   - Connect the other end of the resistor to a ground (GND) rail on the breadboard.
3. **Wire the LED to the Arduino**
   - Use a jumper wire to connect the anode of the LED to digital pin 9 (PWM pin) on the Arduino Uno.
   - Use another jumper wire to connect the ground rail on the breadboard to one of the GND pins on the Arduino.

Step 2: Writing the Code

1. **Open the Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the LED Brightness Control Code**
   - Enter the following code into the Arduino IDE:

```
// LED Brightness Control Example

// Pin number

const int ledPin = 9; // Pin where the LED is connected (PWM pin)

// Sequence of brightness values (0 to 255)

int brightnessSequence[] = {0, 50, 100, 150, 200, 255, 200, 150, 100, 50, 0};

int sequenceLength = sizeof(brightnessSequence) /
sizeof(brightnessSequence[0]);

// Variable to track the current position in the sequence

int currentIndex = 0;

// The setup function runs once when you press reset or power the board

void setup() {

  // Initialize the LED pin as an output

  pinMode(ledPin, OUTPUT);

}

// The loop function runs over and over again forever

void loop() {

  // Set the brightness of the LED

  analogWrite(ledPin, brightnessSequence[currentIndex]);

  // Move to the next value in the sequence

  currentIndex++;
```

```
  // If we have reached the end of the sequence, start over

  if (currentIndex >= sequenceLength) {

    currentIndex = 0;

  }

  // Wait for 500 milliseconds before the next change

  delay(500);

}
```

1. **Save the Code**
   - Save the sketch with an appropriate name, such as "LED_Brightness_Control".

Step 3: Uploading the Code

1. **Connect the Arduino Uno to the Computer**
   - Use the USB cable to connect the Arduino Uno to your computer.
2. **Select the Board and Port**
   - In the Arduino IDE, go to `Tools > Board` and select `Arduino Uno`.
   - Then, go to `Tools > Port` and select the port that your Arduino is connected to (it usually looks like `COM3` or higher on Windows or `/dev/tty.usbmodem` on Mac).
3. **Upload the Code**
   - Click the upload button (right arrow) in the Arduino IDE to compile and upload the code to the Arduino Uno.

Step 4: Observing the Results

1. **Observe the LED Brightness Changes**
   - The LED should gradually change its brightness according to the sequence of values defined in the code. The brightness will increase and decrease smoothly.

Troubleshooting

- If the LED does not respond or change brightness:
  - Ensure all connections are secure and correct.
  - Verify that the LED and resistor are correctly oriented and connected.
  - Make sure the correct board and port are selected in the Arduino IDE.
  - Verify that the code is correctly uploaded (no errors during compilation).

Conclusion

The brightness of an LED is controlled using PWM from a stored sequence of values on an Arduino Uno. This exercise is an introduction to the fundamental concepts of analog output and PWM, which are essential for more complex projects involving varying signal levels.

Experiment 5

Controlling Four LED Patterns Using ASCII Characters Sent from Arduino Serial Monitor (Basic serial data reception using Arduino)

Objective

To create a system that controls four LEDs connected to an Arduino Uno board by sending ASCII characters from the Arduino Serial Monitor to display different LED patterns.

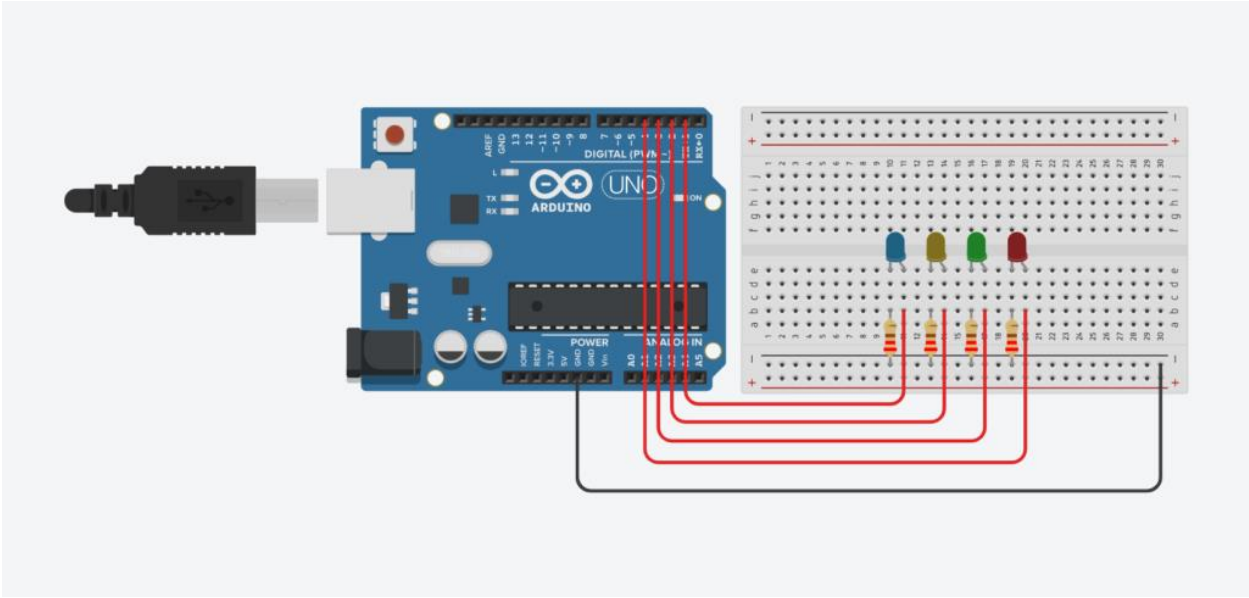
Materials Needed

- Arduino Uno board
- USB cable (Type A to B)
- Breadboard
- Four LEDs (different colors)
- Four 220-ohm resistors
- Jumper wires
- Computer with Arduino IDE installed

Pre-lab Preparation

1. Install the Arduino IDE on your computer if it is not already installed. The software can be downloaded from the official Arduino website.
2. Familiarize yourself with the components: Arduino Uno, breadboard, LEDs, resistors, and jumper wires.

Circuit Diagram



Procedure

Step 1: Setting Up the Hardware

1. **Connect the LEDs to the Breadboard**
   - Place the four LEDs on the breadboard. Each LED represents a different color.
   - Connect the longer leg (anode) of each LED to a separate digital pin on the Arduino (e.g., pins 2, 3, 4, and 5).
   - Connect the shorter leg (cathode) of each LED to a separate 220-ohm resistor.
   - Connect the other end of each resistor to the ground (GND) rail on the breadboard.

Step 2: Writing the Arduino Code

1. **Open the Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the LED Pattern Control Code**
   - Type the following code into the Arduino IDE:

```
// LED Pattern Control Example

// Pin numbers for LEDs
const int led1Pin = 1; // Pin for LED1
const int led2Pin = 2; // Pin for LED2
```

```
const int led3Pin = 3; // Pin for LED3
const int led4Pin = 4; // Pin for LED4

void setup() {
  // Initialize serial communication at 9600 baud
  Serial.begin(9600);

  // Set LED pins as outputs
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  pinMode(led3Pin, OUTPUT);
  pinMode(led4Pin, OUTPUT);
}

void loop() {
  // Check if data is available to read from serial port
  if (Serial.available() > 0) {
    // Read the incoming character from serial port
    char pattern = Serial.read();

    // Based on the received character, turn on/off LEDs to display patterns
    switch (pattern) {
      case '1':
        // Pattern 1: Turn on all LEDs
        digitalWrite(led1Pin, HIGH);
        digitalWrite(led2Pin, HIGH);
        digitalWrite(led3Pin, HIGH);
        digitalWrite(led4Pin, HIGH);
        break;
      case '2':
        // Pattern 2: Turn off all LEDs
        digitalWrite(led1Pin, LOW);
        digitalWrite(led2Pin, LOW);
        digitalWrite(led3Pin, LOW);
        digitalWrite(led4Pin, LOW);
        break;
      case '3':
        // Pattern 3: Alternate blinking of LEDs
        digitalWrite(led1Pin, HIGH);
        digitalWrite(led2Pin, LOW);
        digitalWrite(led3Pin, HIGH);
        digitalWrite(led4Pin, LOW);
        delay(500);
        digitalWrite(led1Pin, LOW);
```

```
      digitalWrite(led2Pin, HIGH);
      digitalWrite(led3Pin, LOW);
      digitalWrite(led4Pin, HIGH);
      delay(500);
      break;
    case '4':
      // Pattern 4: Sequential blinking of LEDs
      digitalWrite(led1Pin, HIGH);
      delay(500);
      digitalWrite(led1Pin, LOW);
      digitalWrite(led2Pin, HIGH);
      delay(500);
      digitalWrite(led2Pin, LOW);
      digitalWrite(led3Pin, HIGH);
      delay(500);
      digitalWrite(led3Pin, LOW);
      digitalWrite(led4Pin, HIGH);
      delay(500);
      digitalWrite(led4Pin, LOW);
      break;
    default:
      // Default case: Turn off all LEDs if unknown character received
      digitalWrite(led1Pin, LOW);
      digitalWrite(led2Pin, LOW);
      digitalWrite(led3Pin, LOW);
      digitalWrite(led4Pin, LOW);
      break;
  }
}
}
```

1. **Save the Code**
   - Save the sketch with an appropriate name, such as "LED_Pattern_fromSerial".

Step 3: Uploading the Code

1. **Connect the Arduino Uno to the Computer**
   - Use the USB cable to connect the Arduino Uno to your computer.
2. **Select the Board and Port**
   - In the Arduino IDE, go to `Tools > Board` and select `Arduino Uno`.
   - Then, go to `Tools > Port` and select the port that your Arduino is connected to (it usually looks like `COM3` or higher on Windows or `/dev/tty.usbmodem` on Mac).

3. **Upload the Code**
    - Click the upload button (right arrow) in the Arduino IDE to compile and upload the code to the Arduino Uno.

Step 4: Controlling LED Patterns from Serial Monitor

1. **Open the Serial Monitor**
    - In the Arduino IDE, go to `Tools > Serial Monitor` or press `Ctrl + Shift + M`.
2. **Send Commands to Control LED Patterns**
    - In the Serial Monitor, type the following commands to control the LED patterns:
        - `1`: Turn on all LEDs.
        - `2`: Turn off all LEDs.
        - `3`: Alternate blinking of LEDs.
        - `4`: Sequential blinking of LEDs.

Troubleshooting

- If the LED patterns do not change as expected:
    - Ensure all connections are secure and correct.
    - Verify that the LEDs are connected to the correct pins on the Arduino.
    - Check the serial communication baud rate settings in both the Arduino sketch and the Serial Monitor.

Conclusion

A system to control four LED patterns using ASCII characters sent from the Arduino Serial Monitor is successfully designed. By sending different commands, various LED patterns are displayed, demonstrating the use of serial communication for real-time control of hardware components.

References

- Arduino Serial Communication
- [Arduino Serial Communication Tutorial](https://www.arduino.cc)

Experiment 6:

Introduction to Thingspeak: Setting Up ThingSpeak Account and Channel Configuration

Objective

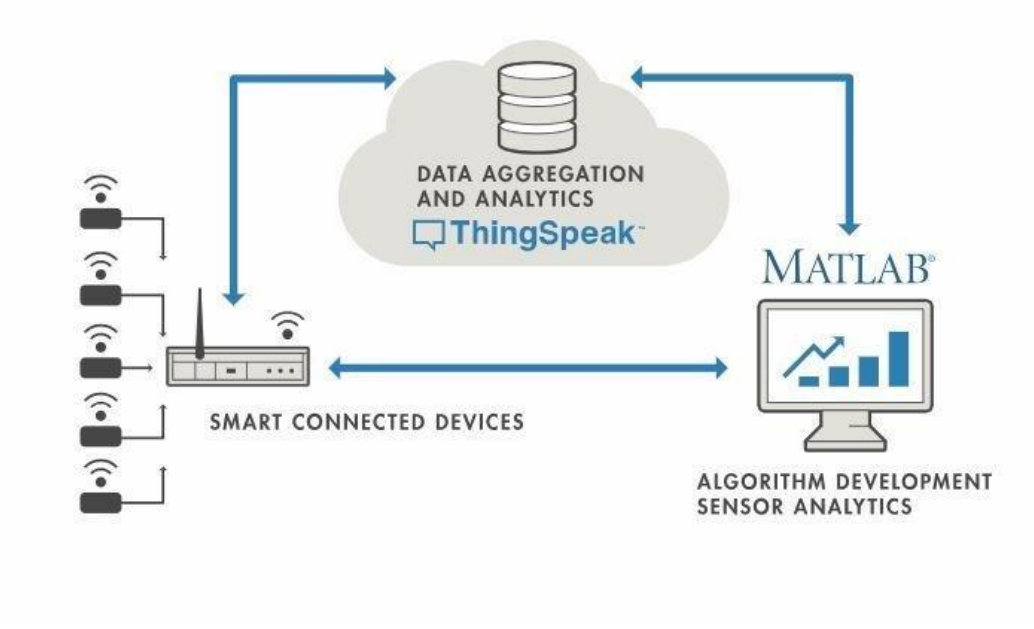
To create a ThingSpeak account, set up a channel, and configure it for data logging and visualization.

Materials Needed

- Computer with internet access
- Web browser

Overview

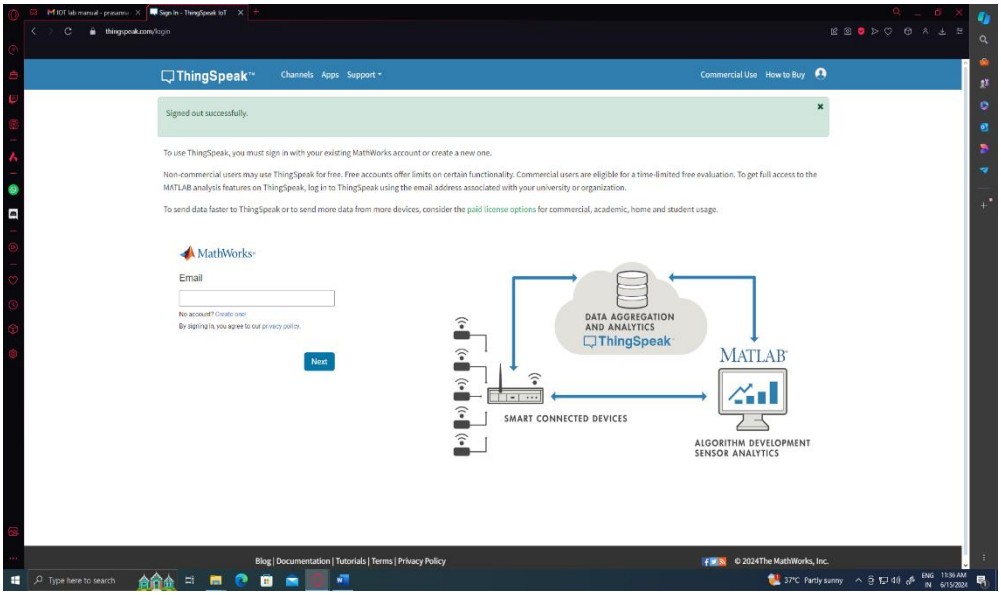
ThingSpeak is a cloud-based IoT platform that allows users to collect, analyze, and visualize data from sensors and other IoT devices. In this lab, we will learn how to create a ThingSpeak account, set up a channel, and configure it for data logging and visualization.



Procedure

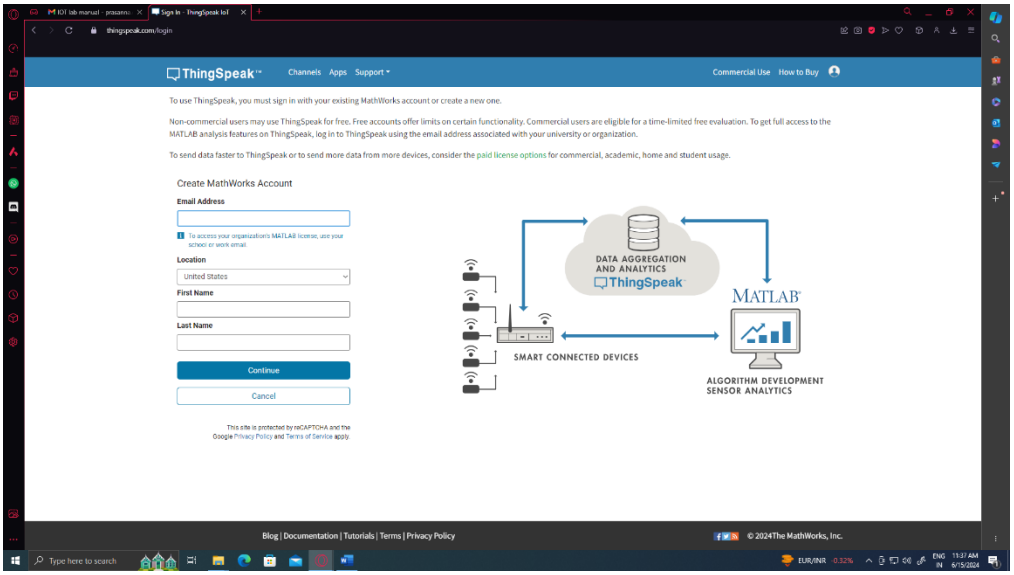
Step 1: Creating a ThingSpeak Account

1. **Open a Web Browser**
   - Launch a web browser on your computer.
2. **Go to ThingSpeak Website**
   - Enter the URL [ThingSpeak] in the address bar and press Enter.



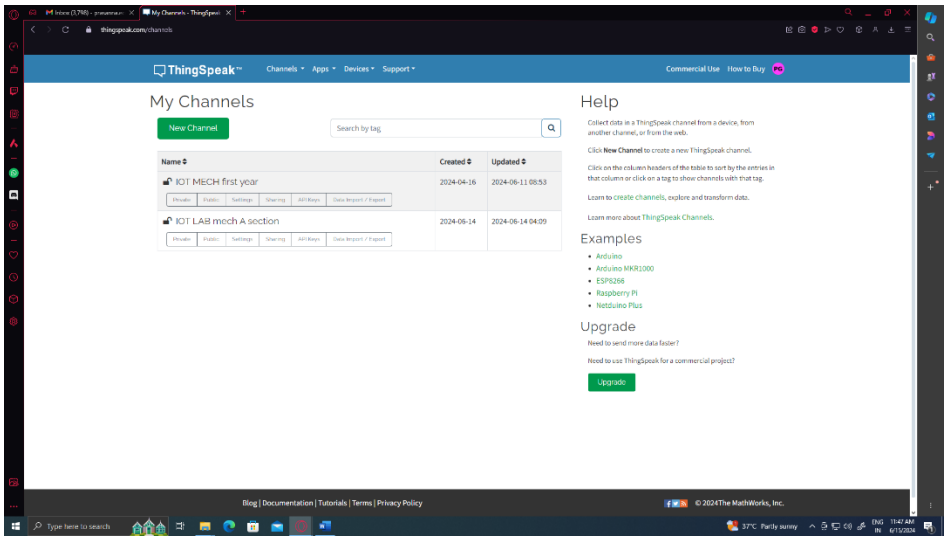
3. **Sign Up for an Account**

- Click on the "Sign Up" link shown near the mathworks logo ThingSpeak landing page.

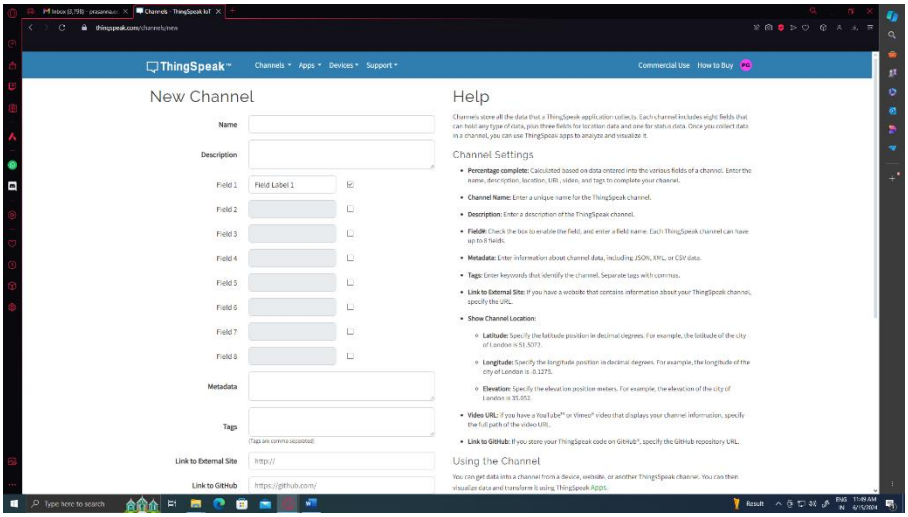


Step 2: Creating a Channel

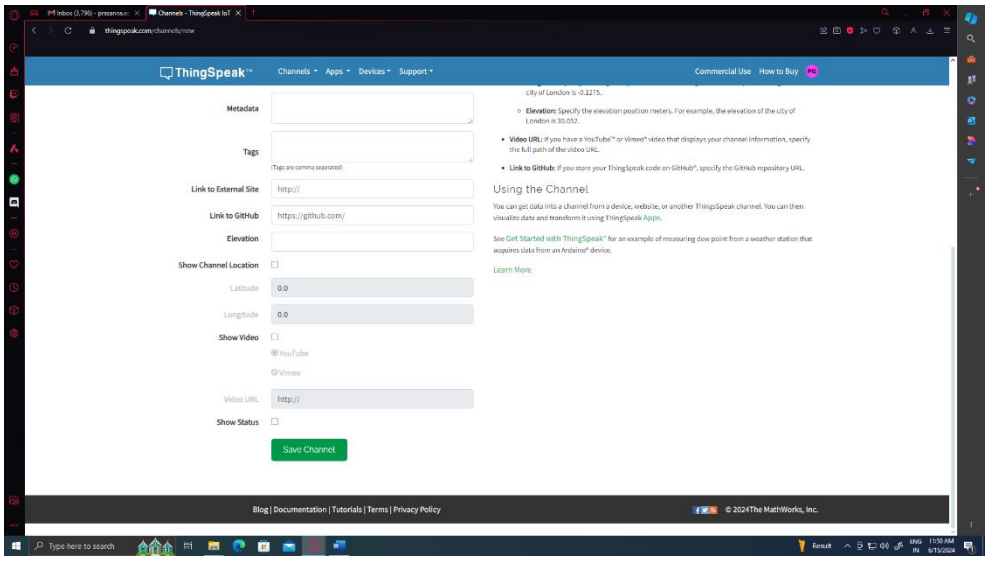
1. **Log In to ThingSpeak**
   - After signing up, log in to your ThingSpeak account using the username and password you created.
2. **Go to Channels**
   - Click on "Channels" in the top menu to navigate to the Channels page.



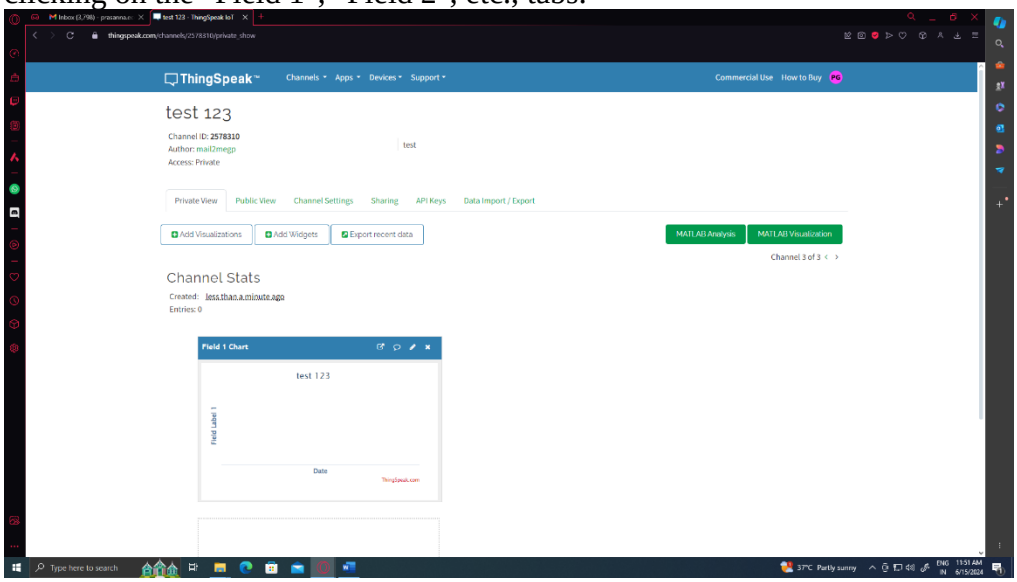
3. **Create a New Channel**
   - Click on the "New Channel" button to create a new channel.



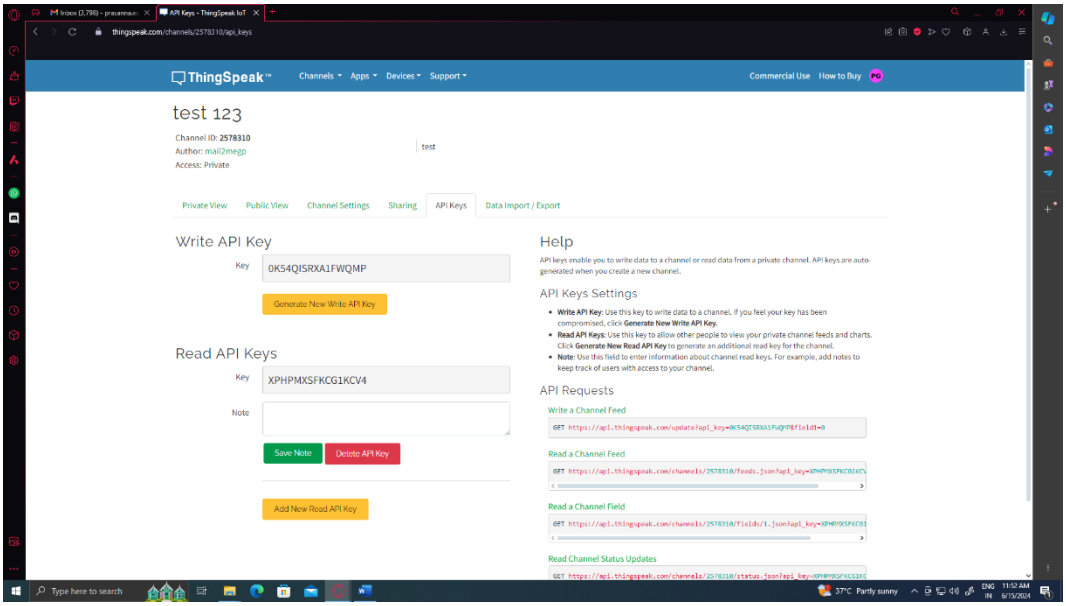
   - Enter the required details such as name, description, and field names for your channel.

- Click on "Save Channel" to create the channel.
- 

Step 3: Configuring the Channel

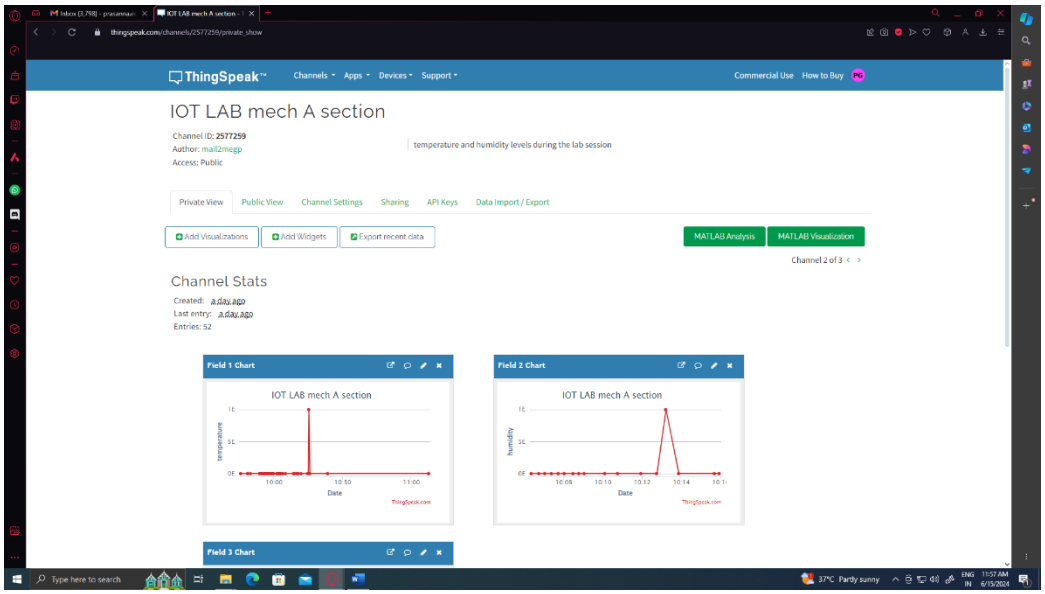
1. **Channel Settings**
   - Once the channel is created, you can configure various settings such as privacy, metadata, and tags by clicking on the "Channel Settings" tab.
2. **Field Settings**
   - Configure field settings such as field labels, units, and data type for each field by clicking on the "Field 1", "Field 2", etc., tabs.
   - 

3. **API Keys**
   - Note down the "Write API Key" from the API Keys tab. This key will be used to send data to your channel.
4. **View and Edit Charts**
   - Click on the "View" tab to view real-time data on charts.
   - You can customize the charts by adding or removing fields and adjusting settings.



Step 4: Testing the Channel (Optional)

1. **Send Test Data**
   - Use the provided API key from another browser address bar in the same PC or a separate PC to send test data to your channel using a HTTP POST/GET request.
   - Verify that the data is being logged and displayed correctly on the charts.



2. **Explore ThingSpeak Features**
   - Explore other features of ThingSpeak such as MATLAB analytics, React, and ThingHTTP to perform advanced actions and integrations.

Conclusion

A ThingSpeak account was successfully created, steps to set up a channel, and configuring it for data logging and visualization were also performed. ThingSpeak provides a user-friendly interface for managing IoT data and offers various tools for analyzing and visualizing data. With the tools learned in this experiments it is possible to integrate sensors or other IoT devices with a ThingSpeak channel for real-world data collection and analysis.

Experiment 7a:

Introduction to NodeMCU: configuring Arduino IDE for NodeMCU operation.

Objective

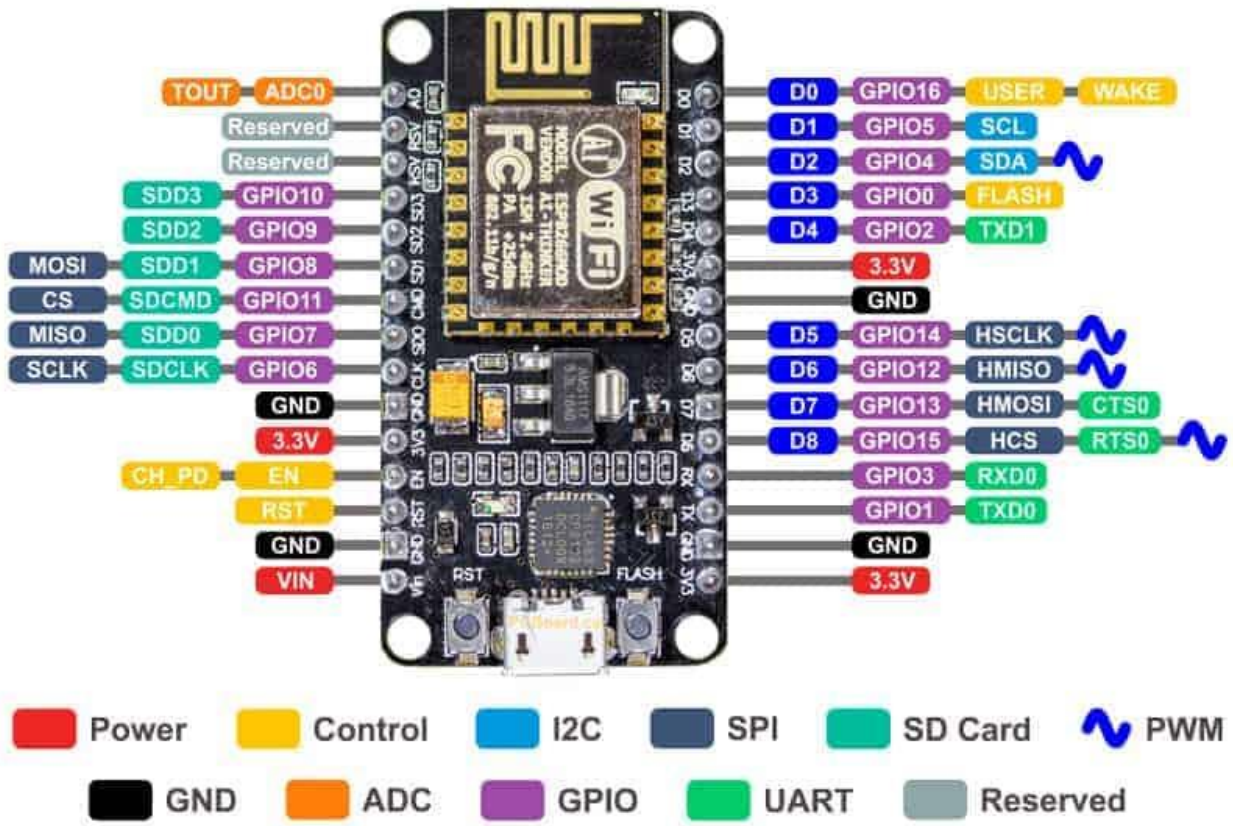
To introduce students to NodeMCU, an open-source firmware and development kit for the Internet of Things (IoT) based on the ESP8266 Wi-Fi module.

Materials Needed

- Computer with internet access
- NodeMCU development board
- Micro-USB cable
- Breadboard
- LEDs
- Resistors
- Jumper wires

Overview

NodeMCU is an open-source firmware and development kit that helps you to prototype your IoT product with Lua script immediately, no need to write any C code. It comes with built-in Wi-Fi support, making it an ideal platform for IoT projects that require wireless connectivity. In this lab, we will learn about the basic features of NodeMCU and how to program it using the Arduino IDE.



Procedure

Step 1: Setting Up NodeMCU

1. **Connect NodeMCU to Computer**
   - Connect the NodeMCU development board to your computer using the micro-USB cable.
2. **Install CH340 Driver (if needed)**
   - If your computer does not recognize the NodeMCU board, you may need to install the CH340 driver. Search online for the appropriate driver for your operating system and follow the installation instructions.

Step 2: Setting Up Arduino IDE for NodeMCU

1. **Install Arduino IDE**
   - If you haven't already, download and install the Arduino IDE from the official Arduino website (Arduino IDE).
2. **Install ESP8266 Board Package**
   - Open the Arduino IDE.

- Go to `File > Preferences`.
- In the "Additional Board Manager URLs" field, add the following URL:

  ```
  bash
  Copy code
  http://arduino.esp8266.com/stable/package_esp8266com_index.json
  ```

- Click "OK" to close the Preferences window.
- Go to `Tools > Board > Boards Manager....`
- Search for "esp8266" and install the "esp8266" board package.

3. **Select NodeMCU Board**
   - Go to `Tools > Board` and select the appropriate NodeMCU board (e.g., NodeMCU 1.0 (ESP-12E Module)).
4. **Select COM Port**
   - Go to `Tools > Port` and select the COM port to which your NodeMCU is connected.

Step 3: Testing NodeMCU with Blink Program

1. **Open Blink Example**
   - Go to `File > Examples > 01.Basics > Blink` to open the Blink example sketch.
2. **Modify Sketch for NodeMCU**
   - In the Blink sketch, modify the LED pin number to `D0` (which corresponds to GPIO16 on the NodeMCU).

     ```
     cpp
     Copy code
     // Pin number for the built-in LED on NodeMCU
     const int ledPin = D0;
     ```

3. **Upload Sketch**
   - Click the upload button (right arrow) to compile and upload the sketch to the NodeMCU.
4. **Verify Blinking**
   - After uploading, the built-in LED on the NodeMCU should start blinking.

Step 4: Experiment with External Components (Optional)

1. **Connect External Components**
   - Use the breadboard, LEDs, resistors, and jumper wires to connect external components to the NodeMCU.
2. **Write and Upload Custom Sketch**
   - Write your own sketch to control the external components (e.g., blinking multiple LEDs).
   - Upload the sketch to the NodeMCU and observe the behavior of the external components.

Step 5: Explore NodeMCU Capabilities

1. **Learn Lua Programming (Optional)**
   - NodeMCU supports programming in Lua, a lightweight scripting language. Explore Lua programming to take full advantage of NodeMCU's capabilities.
2. **Explore Wi-Fi Connectivity**
   - NodeMCU comes with built-in Wi-Fi support. Experiment with connecting NodeMCU to a Wi-Fi network and implementing IoT projects that require wireless communication.

Conclusion

You have successfully set up NodeMCU and programmed it using the Arduino IDE. NodeMCU offers a versatile platform for IoT prototyping and development, with built-in Wi-Fi support and compatibility with the Arduino ecosystem. You can now explore more advanced projects and experiments with NodeMCU to build your IoT applications.

References

- NodeMCU Documentation
- ESP8266 Arduino Core Documentation

Experiment 7b

NodeMCU Application for Collecting DHT11 Sensor data and uploading to ThingSpeak Channel

Objective

To develop a basic NodeMCU application that collects temperature and humidity data from a DHT11 sensor and sends the data to a ThingSpeak channel using a WiFi hotspot.

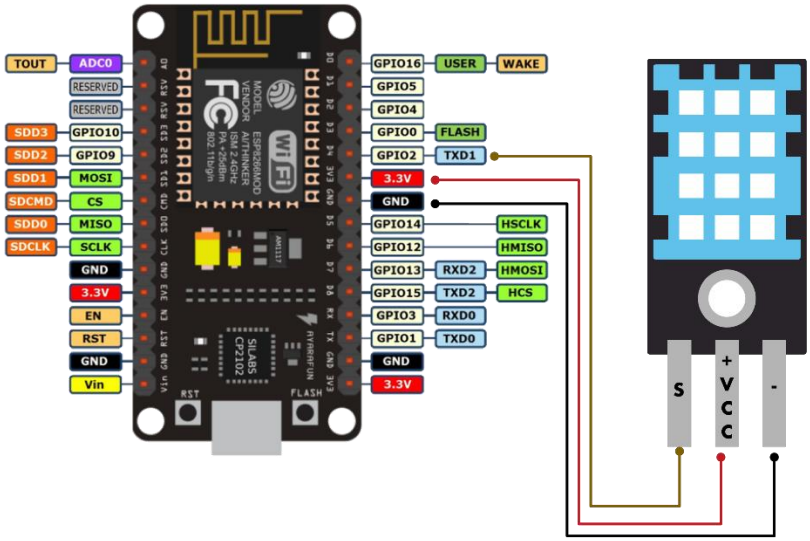
Components Needed

- NodeMCU development board
- DHT11 temperature and humidity sensor

- Micro-USB cable
- Breadboard
- Jumper wires

Overview

In this lab, we will create a NodeMCU application that reads temperature and humidity data from a DHT11 sensor, connects to a WiFi hotspot, and sends the data to a ThingSpeak channel for logging and visualization.



Procedure

Step 1: Setting Up Hardware

1. **Connect DHT11 Sensor to NodeMCU**
   - Connect the DHT11 sensor to the NodeMCU as follows:
     - DHT11 VCC pin to NodeMCU 3.3V pin
     - DHT11 DATA pin to NodeMCU D1 pin (GPIO5)
     - DHT11 GND pin to NodeMCU GND pin

Step 2: Setting Up Arduino IDE for NodeMCU

1. **Install Arduino IDE**
   - If you haven't already, download and install the Arduino IDE from the official Arduino website (Arduino IDE).
2. **Install ESP8266 Board Package**
   - Open the Arduino IDE.
   - Go to `File > Preferences`.
   - In the "Additional Board Manager URLs" field, add the following URL:

```
bash
Copy code
http://arduino.esp8266.com/stable/package_esp8266com_index.json
```

- Click "OK" to close the Preferences window.
- Go to `Tools > Board > Boards Manager....`
- Search for "esp8266" and install the "esp8266" board package.

3. **Select NodeMCU Board**
   - Go to `Tools > Board` and select the appropriate NodeMCU board (e.g., NodeMCU 1.0 (ESP-12E Module)).
4. **Select COM Port**
   - Go to `Tools > Port` and select the COM port to which your NodeMCU is connected.

Step 3: Writing the Arduino Sketch

1. **Open Arduino IDE**
   - Launch the Arduino IDE on your computer.
2. **Write the Arduino Sketch**
   - Write the Arduino sketch to read data from the DHT11 sensor and send it to a ThingSpeak channel. You can use the following example code:

```
#include <DHT.h>

#include <Adafruit_Sensor.h>

#include <ESP8266WiFi.h>

String apiKey = "WGVG2C223R81XWP4";     //  Enter your Write API key from ThingSpeak

const char *ssid =  "Muruga";     // replace with your wifi ssid and wpa2 key

const char *pass =  "ponds123";

const char* server = "api.thingspeak.com";

#define DHTPIN 2         //pin where the dht11 is connected

DHT dht(DHTPIN, DHT11);

WiFiClient client;

void setup()

{
```

```
  Serial.begin(115200);
  delay(10);
  dht.begin();
  Serial.println("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, pass);
 while (WiFi.status() != WL_CONNECTED)
 {
     delay(500);
     Serial.print(".");
 }
 Serial.println("");
 Serial.println("WiFi connected");
}
void loop()
{

 float h = dht.readHumidity();
 float t = dht.readTemperature();

     if (isnan(h) || isnan(t))
       {
         Serial.println("Failed to read from DHT sensor!");
          return;
       }
           if (client.connect(server,80))   //   "184.106.153.149" or api.thingspeak.com
          {

              String postStr = apiKey;
```

```
                postStr +="&field1=";
                postStr += String(t);
                postStr +="&field2=";
                postStr += String(h);
                postStr += "\r\n\r\n";

                client.print("POST /update HTTP/1.1\n");
                client.print("Host: api.thingspeak.com\n");
                client.print("Connection: close\n");
                client.print("X-THINGSPEAKAPIKEY: "+apiKey+"\n");
                client.print("Content-Type: application/x-www-form-urlencoded\n");
                client.print("Content-Length: ");
                client.print(postStr.length());
                client.print("\n\n");
                client.print(postStr);

                Serial.print("Temperature: ");
                Serial.print(t);
                Serial.print(" degrees Celcius, Humidity: ");
                Serial.print(h);
                Serial.println("%. Send to Thingspeak.");
            }
        client.stop();

        Serial.println("Waiting...");

  // thingspeak needs minimum 15 sec delay between updates
  delay(1000);
}
```

3. **Configure Sketch**
    - Replace `YOUR_WIFI_SSID`, `YOUR_WIFI_PASSWORD`, `YOUR_CHANNEL_ID`, and `YOUR_WRITE_API_KEY` with your WiFi credentials and ThingSpeak channel details.

Step 4: Upload Sketch to NodeMCU

1. **Upload Sketch**
    - Click the upload button (right arrow) in the Arduino IDE to compile and upload the sketch to the NodeMCU.
2. **Monitor Serial Output**
    - Open the serial monitor in the Arduino IDE to monitor the NodeMCU's serial output. You should see temperature and humidity data being printed.

Step 5: View Data on ThingSpeak

1. **View ThingSpeak Channel**
    - Log in to your ThingSpeak account and navigate to the channel where you configured the NodeMCU to send data.
2. **Visualize Data**
    - ThingSpeak provides various visualization tools to plot and analyze the data sent by the NodeMCU. Explore

Conclusion

You have successfully created a NodeMCU application that collects temperature and humidity data from a DHT11 sensor and sends the data to a ThingSpeak channel via a WiFi hotspot. This lab demonstrates the capabilities of NodeMCU in IoT applications for data collection and visualization.

References

- NodeMCU Documentation
- ESP8266 Arduino Core Documentation